Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií

Protokol k zadaniu

DBS — Databázové technológie

Zadanie 3: Implementácia fyzického modelu a procesov

Reštauračný systém

Študent: Dávid Boldog, Tomáš Bubeník
Študijná skupina: Štvrtok 11:00
Cvičiaci: Ing. Matej Petrík
Akademický rok: 2025/2026
Dátum: 3. mája 2026

Obsah

2 Zmeny oproti Zadaniu 1 **2**

3 Fyzický model - implementácia **3**

4 Seed dáta **5**

5 Proces 1 - Vytvorenie dine-in objednávky **6**

6 Proces 2 - Výkonnostný report čašníkov **8**

7 Výkonnosť a indexy **10**

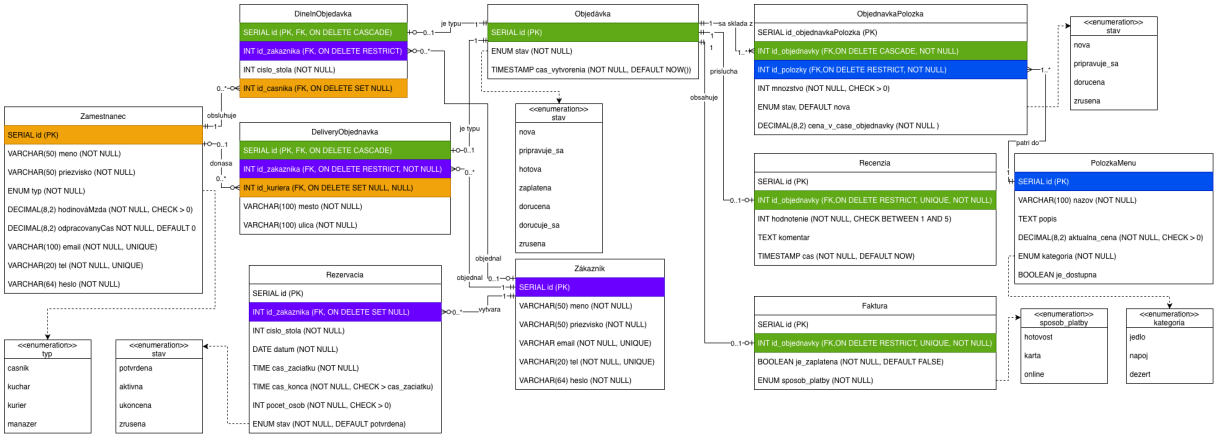
8 Zhodnotenie **13**

2 Zmeny oproti Zadaniu 1

Model ostal tematicky aj vzťahovo rovnaký ako v Zadaní 1: jadrom systému je spoločná tabuľka `Objednavka`, ktorú špecializujú tabuľky `DineInObjednavka` a `DeliveryObjednavka`. Pri implementácii fyzického modelu sme doplnili konkrétne PostgreSQL dátové typy, ENUM hodnoty, referenčné akcie a indexy potrebné pre procesy v tomto zadaní.

Najdôležitejšie spresnenia voči návrhu sú:

- stavové atribúty sú implementované ako PostgreSQL ENUM typy,
- historická cena položky objednávky je uložená v `ObjednavkaPolozka.cena_v_case_objednavky`,
- prekrývanie rezervácií sa nevynucuje cez `CHECK`, pretože ide o medziriadkové pravidlo; procesy ho kontrolujú dotazom,
- vzťahy `Objednavka`–`Faktura` a `Objednavka`–`Recenzia` sú spresnené ako voliteľné 1:1; oproti pôvodnému diagramu sme opravili kardinalitu na strane faktúry a recenzie z `0..*` na `0..1`,
- proces vytvorenia dine-in objednávky po úspešnom overení prepne rezerváciu do stavu `aktivna`.



Obr. 1: Aktuálny fyzický model databázy

3 Fyzický model - implementácia

Fyzický model je implementovaný v súbore `schema.sql`. Skript najprv odstráni existujúce tabuľky a ENUM typy, následne vytvorí všetky typy, tabuľky, integritné obmedzenia a vlastné indexy.

Tabuľka	Účel
`Zamestnanec`	Evidencia čašníkov, kuchárov, kuriérov a manažérov.
`Zakaznik`	Registrovaní zákazníci pre rezervácie, dine-in a donášku.
`PolozkaMenu`	Aktuálna ponuka jedál, nápojov a dezertov.
`Objednavka`	Spoločná entita pre všetky objednávky.
`DineInObjednavka`	Špecializácia objednávky pri stole.
`DeliveryObjednavka`	Špecializácia donáškovej objednávky.
`ObjednavkaPolozka`	M:N vzťah objednávky a menu položiek so snímkou ceny.
`Rezervacia`	Rezervácia konkrétneho stola v časovom intervale.
`Faktura`	Platobný doklad k objednávke.
`Recenzia`	Jedno hodnotenie k jednej objednávke.

Tabuľka 1: Zoznam fyzických tabuliek

Integritné obmedzenia

Model používa primárne kľúče typu `SERIAL`, cudzie kľúče, `NOT NULL`, `UNIQUE`, `CHECK` a `DEFAULT`. Netriviálne pravidlá sú:

- `email` a `tel` sú unikátne pri zákazníkoch aj zamestnancoch,
- ceny, mzdy, množstvá a počet osôb musia byť kladné,
- hodnotenie recenzie je obmedzené na interval 1 až 5,
- koniec rezervácie musí byť po začiatku rezervácie,
- `Faktura.id_objednavky` a `Recenzia.id_objednavky` sú unikátne, teda k objednávke existuje najviac jedna faktúra a najviac jedna recenzia.

Väzba na biznis pravidlá zo Zadania 1

Fyzický model a seed dáta boli skontrolované voči pravidlám zo Zadania 1. Jednoznačnosť kontaktov zákazníkov a zamestnancov vynucujú `UNIQUE` obmedzenia na `email` a `tel` a generátor používa unikátne hodnoty. Kladné ceny, mzdy, množstvá a počet osôb sú chránené `CHECK` obmedzeniami a seed ich generuje iba v povolených rozsahoch. Delivery objednávka má povinné mesto a ulicu, takže ju nemožno vložiť bez doručovacej adresy.

Dostupnosť menu položiek je chránená v procese vytvorenia objednávky cez filter `je_dostupna = TRUE`; seed zároveň vyberá do objednávok iba dostupné položky. Historická cena je uložená priamo v `ObjednavkaPolozka.cena_v_case_objednavky`, takže zmeny aktuálnej ceny menu neskreslia staré objednávky. Jedinečnosť faktúry a recenzie na objednávku vynucuje `UNIQUE` na `id_objednavky`; seed negeneruje duplicitné faktúry ani duplicitné recenzie.

Pravidlo neprekrývania rezervácií je medziriadkové. V schéme je preto priamo vynútený aspoň platný časový interval rezervácie (`cas_konca > cas_zaciatku`) a operačný proces pracuje iba s jednou potvrdenou rezerváciou pre konkrétny stôl, zákazníka, dátum a čas. Seed bol upravený tak, aby sa potvrdené a aktívne rezervácie rovnakého stola časovo neprekrývali.

ON DELETE stratégie

- `ON DELETE CASCADE` je použité medzi `Objednavka` a jej detailmi, aby sa neosireli špecializačné a položkové záznamy.
- `ON DELETE RESTRICT` chráni historické objednávky pred zmazaním zákazníka alebo položky menu, na ktorú sa odkazujú.
- `ON DELETE SET NULL` sa používa pri voliteľných väzbách na zamestnanca alebo zákazníka, kde chceme zachovať historický záznam aj po odstránení osoby.

4 Seed dáta

Seed dáta sú generované skriptom `seed/seed.py` pomocou knižnice Faker so slovenskou lokalizáciou `sk_SK`. Generovanie je reprodukovateľné, pretože skript používa fixný seed `SEED = 42`. Výstupom je súbor `seed/seed.sql`, ktorý vkladá dáta do všetkých hlavných tabuliek a na konci nastaví hodnoty sekvencií cez `setval`. Reprodukcia seedu je popísaná v `seed/README.md`.

Tabuľka	**Počet záznamov**
`Zamestnanec`	25
`Zakaznik`	600
`PolozkaMenu`	50
`Rezervacia`	1 500
`Objednavka`	4 000
`DineInObjednavka`	2 338
`DeliveryObjednavka`	1 662
`ObjednavkaPolozka`	10 764
`Faktura`	3 621
`Recenzia`	1 177
Spolu	**25 737**

Tabuľka 2: Počty záznamov po seede

Distribúcia dát je biznisovo motivovaná: objednávky sú rozdelené približne na 60 % dine-in a 40 % delivery, zákazníci sú vyberaní váženou náhodnosťou, takže časť zákazníkov objednáva často a časť vôbec. Rezervácie sú rozložené medzi minulosť, aktuálny deň a budúcnosť, pričom budúce rezervácie neobsahujú dokončené stavy.

Konzistencia je zabezpečená poradím generovania dát. Najprv vznikajú referenčné entity (`Zakaznik`, `Zamestnanec`, `PolozkaMenu`), potom rezervácie, objednávky, špecializácie objednávok, položky, faktúry a recenzie. Generátor vyberá iba existujúce cudzie kľúče a rešpektuje roly zamestnancov pri čašníkoch a kuriéroch.

5 Proces 1 - Vytvorenie dine-in objednávky

Popis procesu

Proces pokrýva príchod zákazníka s rezerváciou a vytvorenie objednávky pri stole. Vstupom sú číslo stola, zákazník, dátum a čas príchodu, čašník a tri objednávané položky s množstvami. Výstupom je detail vytvorenej objednávky alebo zoznam validačných chýb.

Proces overuje, že pre daný stôl, zákazníka, dátum a čas existuje potvrdená rezervácia a že všetky objednávané položky menu existujú, sú dostupné a majú kladné množstvo. Ak kontroly prejdú, v jednom SQL príkaze sa aktivuje rezervácia a vložia sa záznamy do `Objednavka`, `DineInObjednavka` a `ObjednavkaPolozka`. Referenčnú integritu zákazníka a čašníka následne chránia cudzie kľúče fyzického modelu.

Väzba na scenár zo Zadania 1

Proces priamo nadväzuje na Scenár č. 1 - Rezervácia stola a príchod do reštaurácie a na začiatok Scenára č. 2 - Objednanie jedla, platba a recenzia. Zo Scenára č. 1 preberá overenie rezervácie pri príchode zákazníka a jej prepnutie do stavu `aktivna`; zo Scenára č. 2 preberá vytvorenie dine-in objednávky, kontrolu dostupných položiek menu a uloženie snímky ceny.

SQL implementácia

Implementácia je v súbore `queries.sql`. Proces nepoužíva dočasné tabuľky, PL/pgSQL blok `DO`, vlastné uložené funkcie, procedúry, triggery ani kurzory. Celý zápis je jeden SQL statement. CTE sa používajú iba tam, kde sú potrebné na spoločné vstupy alebo na reťazenie `UPDATE/INSERT ... RETURNING`; kontrola položiek menu je riešená cez vnorené subqueries. Nasledujúci výpis je skrátená kostra; úplná spustiteľná verzia je v súbore `queries.sql`.

```sql
WITH
params AS (...),
input_items(polozka_id, mnozstvo) AS (...),
valid_reservation AS (...),
activated_reservation AS (
    UPDATE Rezervacia
    SET stav = 'aktivna'
    WHERE NOT EXISTS (
        SELECT 1
        FROM input_items
        WHERE mnozstvo <= 0 OR polozka_nie_je_dostupna
    )
    RETURNING id
),
inserted_order AS (
    INSERT INTO Objednavka (cas_vytvorenia)
    SELECT datum + cas_prichodu
    FROM params
    WHERE EXISTS (SELECT 1 FROM activated_reservation)
    RETURNING id, stav, cas_vytvorenia
),
inserted_dine_in AS (...),
inserted_items AS (...)
SELECT *
FROM vysledok_procesu;
```

Ukážka spustenia

Testovacie parametre v `queries.sql`:

- stôl: 8,
- zákazník: `id = 320`,
- čas príchodu: `2026-05-03 13:15`,
- čašník: `id = 13`,
- položky: `12 x 1`, `17 x 2`, `24 x 2`.

Položka	**Stav**	**Množstvo**	**Subtotal**
Pizza Margherita	nova	1	13.30
Kurací burger	nova	2	31.82
Cappuccino	nova	2	7.64
Spolu			**52.76**

Tabuľka 3: Očakávaný výstup procesu 1 nad seed dátami

Zdôvodnenie korektnosti

Objednávka sa vloží iba vtedy, keď CTE `activated_reservation` vráti riadok. Toto CTE vráti riadok len pri nájdenej potvrdenej rezervácii a zároveň pri absencii neplatnej alebo nedostupnej položky. Nadväzujúce `INSERT` CTE sú preto prázdne, ak validácia neprejde. Keďže všetky zápisy sú súčasťou jedného SQL statementu, nevznikne situácia, že by sa vytvorila iba časť objednávky. Snímka ceny sa berie z aktuálnej ceny v `PolozkaMenu` v momente vytvorenia objednávky.

Indexy podporujúce proces

- `idx_rezervacia_stol_datum` podporuje hľadanie potvrdenej rezervácie podľa stola, dátumu a stavu.
- `idx_polozka_dostupna` podporuje filtrovanie dostupného menu; pri kontrole konkrétnej položky podľa ID môže databáza použiť aj primárny kľúč `PolozkaMenu.id`.

6 Proces 2 - Výkonnostný report čašníkov

Popis procesu

Manažér chce za zvolený rok vidieť výkon jednotlivých čašníkov. Report pracuje iba so zaplatenými dine-in objednávkami, teda so záznamami, ku ktorým existuje faktúra s `je_zaplatena = TRUE`. Výstup obsahuje počet objednávok, celkovú tržbu, priemernú hodnotu objednávky, priemerné hodnotenie, poradie podľa tržby, percentuálny podiel a kumulatívnu tržbu.

Väzba na scenár zo Zadania 1

Proces využíva dáta, ktoré vznikajú v Scenári č. 2 - Objednanie jedla, platba a recenzia: dine-in objednávky, položky objednávky, faktúry a recenzie. Zároveň nadväzuje na manažérsku rolu zo špecifikácie Zadania 1, kde manažér generuje prevádzkové reporty a potrebuje prehľad o výkone čašníkov.

SQL implementácia

Proces používa pohľad `v_dine_in_trzby`, vnorené subqueries, agregácie a window funkcie. Rok sa filtruje cez rozsah dátumov, nie cez `EXTRACT(YEAR ...)`, aby databáza vedela efektívnejšie použiť index na `Objednavka.cas_vytvorenia`.

```sql
CREATE OR REPLACE VIEW v_dine_in_trzby AS
SELECT o.id, o.cas_vytvorenia, d.id_casnika,
       SUM(op.mnozstvo * op.cena_v_case_objednavky) AS trzba_objednavky
FROM Objednavka o
JOIN Faktura f ON f.id_objednavky = o.id AND f.je_zaplatena = TRUE
JOIN DineInObjednavka d ON d.id = o.id
JOIN ObjednavkaPolozka op ON op.id_objednavky = o.id
                         AND op.stav <> 'zrusena'
WHERE o.stav <> 'zrusena'
GROUP BY o.id, o.cas_vytvorenia, d.id_casnika;

SELECT w.id_casnika, z.meno, z.priezvisko,
       w.pocet_objednavok, w.celkova_trzba,
       w.avg_hodnotenie, w.poradie
FROM (
    SELECT s.*, h.avg_hodnotenie,
           RANK() OVER (ORDER BY s.celkova_trzba DESC) AS poradie
    FROM (
        SELECT id_casnika, COUNT(*) AS pocet_objednavok,
               SUM(trzba_objednavky) AS celkova_trzba
        FROM v_dine_in_trzby
        WHERE cas_vytvorenia >= '2026-01-01'
          AND cas_vytvorenia < '2027-01-01'
        GROUP BY id_casnika
    ) s
    LEFT JOIN (
        SELECT d.id_casnika, AVG(r.hodnotenie) AS avg_hodnotenie
        FROM Recenzia r
        JOIN Objednavka o ON o.id = r.id_objednavky
        JOIN DineInObjednavka d ON d.id = o.id
        WHERE o.cas_vytvorenia >= '2026-01-01'
          AND o.cas_vytvorenia < '2027-01-01'
        GROUP BY d.id_casnika
    ) h ON h.id_casnika = s.id_casnika
) w
JOIN Zamestnanec z ON z.id = w.id_casnika;
```

Ukážka výstupu

Poradie	Čašník	Obj.	Tržba	Priemer	Hodn.
1	Kristián Kučerová	47	3001.85	63.87	4.13
2	Ľubomíra Miklošová	46	2797.38	60.81	4.00
3	Metod Huba	38	2528.98	66.55	3.75
4	Leonard Macháčeková	34	2460.25	72.36	3.75
5	Irena Janečková	35	2435.39	69.58	3.93

Tabuľka 4: Prvých päť riadkov reportu pre rok 2026

Zdôvodnenie korektnosti

Tržba vzniká ako súčet `mnozstvo * cena_v_case_objednavky`, teda používa historickú cenu uloženú v objednávke, nie aktuálnu cenu menu. Zrušené objednávky a zrušené položky sa do reportu nezapočítavajú. Join na `Faktura` zabezpečí, že report reprezentuje reálne zaplatené tržby.

Indexy podporujúce proces

- `idx_objednavka_cas_vytvorenia` podporuje rozsahový filter zvoleného roka.
- `idx_faktura_zaplatena` podporuje výber zaplatených faktúr; na aktuálnom seede však plánovač volí sekvenčný sken, keďže takmer všetky faktúry sú zaplatené.
- `idx_objednavkapolozka_objednavka` podporuje agregáciu položiek podľa objednávky.

Kontrola cez `EXPLAIN (ANALYZE, BUFFERS)` nad seed dátami potvrdila použitie indexu nad dátumom vytvorenia objednávky pri filtrovaní roka 2026 a indexu nad cudzím kľúčom položiek pri pripájaní položiek objednávky. Pri tabuľke `Faktura` plánovač zvolil sekvenčný sken, pretože filter `je_zaplatena = TRUE` v seede vyberá takmer všetky riadky.

7 Výkonnosť a indexy

PostgreSQL automaticky vytvára indexy pre `PRIMARY KEY` a `UNIQUE` obmedzenia. Tieto indexy sa do požadovaného minima nepočítajú. V schéme preto vytvárame päť vlastných indexov, ktoré priamo podporujú dva implementované procesy.

idx_rezervacia_stol_datum

```
CREATE INDEX idx_rezervacia_stol_datum
    ON Rezervacia (cislo_stola, datum, stav);
```

Index podporuje proces 1, konkrétne validáciu rezervácie pred vytvorením dine-in objednávky. Dopyt filtruje tabuľku `Rezervacia` podľa `cislo_stola`, `datum` a `stav`. Bez indexu musí databáza prejsť všetky rezervácie sekvenčným skenom a až potom odfiltrovať konkrétny stôl, dátum a stav. Index má odstrániť práve tento sekvenčný sken nad tabuľkou `Rezervacia`; zvyšné podmienky na zákazníka a časový interval sa potom aplikujú iba na malú množinu kandidátov.

idx_polozka_dostupna

```
CREATE INDEX idx_polozka_dostupna
    ON PolozkaMenu (je_dostupna);
```

Index podporuje proces 1 pri validácii, že objednávané položky menu existujú a sú dostupné. Dopyt aj pohľad `v_dostupne_polozky` používajú podmienku `je_dostupna = TRUE`. Pri overení konkrétnej položky objednávky je však v predikáte aj `PolozkaMenu.id`, takže databáza môže namiesto tohto indexu použiť primárny kľúč nad `id` a dostupnosť skontrolovať až filtrom. Index je preto najvhodnejší pre časté filtrovanie dostupného menu; pri malej tabuľke menu môže optimalizátor zvoliť aj sekvenčný sken.

idx_objednavkapolozka_objednavka

```
CREATE INDEX idx_objednavkapolozka_objednavka
    ON ObjednavkaPolozka (id_objednavky);
```

Index podporuje najmä proces 2 pri agregácii tržby objednávky, kde sa položky pripájajú cez `ObjednavkaPolozka.id_objednavky`. V aktuálnej verzii procesu 1 sa detail práve vytvorenej objednávky skladá z CTE `inserted_items`, takže tento index nie je potrebný na finálny výstup procesu 1. Ostáva však dôležitý pri bežnom neskoršom načítaní detailu objednávky podľa `id_objednavky` a pri analytickom reporte. PostgreSQL nevytvára index automaticky na stĺpci cudzieho kľúča, preto je tento index vlastný.

idx_objednavka_cas_vytvorenia

```
CREATE INDEX idx_objednavka_cas_vytvorenia
    ON Objednavka (cas_vytvorenia);
```

Index podporuje proces 2, kde manažérsky report filtruje objednávky za zvolený rok. Dopyt je zámerne zapísaný ako rozsah:

```
WHERE o.cas_vytvorenia >= (current_setting('myapp.p_rok') || '-01-01')::TIMESTAMP
  AND o.cas_vytvorenia < ((current_setting('myapp.p_rok')::INT + 1)::TEXT || '
      -01-01')::TIMESTAMP
```

Takýto zápis je vhodný pre B-tree index nad `cas_vytvorenia`. Index má obmedziť sekvenčné skenovanie všetkých objednávok pri ročných reportoch. Keby bol filter zapísaný ako `EXTRACT(YEAR FROM cas_vytvorenia)`, databáza by musela vypočítať výraz pre každý riadok a tento index by sa používal horšie alebo vôbec.

`idx_faktura_zaplatena`

```
CREATE INDEX idx_faktura_zaplatena
    ON Faktura (je_zaplatena, id_objednavky);
```

Index podporuje proces 2. Report má počítať iba reálne tržby, preto pohľad `v_dine_in_trzby` joinuje tabuľku `Faktura` a filtruje `je_zaplatena = TRUE`. Prvý stĺpec indexu podporuje filter zaplatených faktúr, druhý stĺpec podporuje následný join na `Objednavka.id`. Na aktuálnom seede je však hodnota `je_zaplatena = TRUE` veľmi málo selektívna, pretože zaplatených je 3602 z 3621 faktúr. Reálny plán preto používa `Seq Scan on faktura`. Index ostáva opodstatnený pre väčšiu alebo vyváženejšiu produkčnú databázu, kde filter zaplatených faktúr môže znížiť počet čítaných riadkov.

EXPLAIN porovnanie

Porovnanie sme spravili pre proces 1, konkrétne pre validáciu rezervácie. Test bol spustený nad naplnenou databázou so seed dátami. Najprv sme index `idx_rezervacia_stol_datum` odstránili a spustili plán:

```
DROP INDEX IF EXISTS idx_rezervacia_stol_datum;

EXPLAIN (ANALYZE, BUFFERS)
SELECT r.id, r.id_zakaznika, r.cislo_stola, r.datum,
       r.cas_zaciatku, r.cas_konca, r.stav
FROM Rezervacia r
WHERE r.cislo_stola = 8
  AND r.id_zakaznika = 320
  AND r.datum = DATE '2026-05-03'
  AND TIME '13:15' BETWEEN r.cas_zaciatku AND r.cas_konca
  AND r.stav = 'potvrdena'
LIMIT 1;
```

Výrez plánu bez indexu:

```
Limit (actual time=0.237..0.237 rows=1 loops=1)
  Buffers: shared hit=13
  -> Seq Scan on rezervacia r
        Rows Removed by Filter: 1376
        Buffers: shared hit=13
Planning Time: 2.329 ms
Execution Time: 0.254 ms
```

Bez indexu databáza použila `Seq Scan on rezervacia`. To znamená, že prešla všetkých 1 500 rezervácií a v tomto teste zahodila 1 376 riadkov filtrom, kým našla vyhovujúcu rezerváciu.

Následne sme index vytvorili znova a spustili ten istý dopyt:

```
CREATE INDEX idx_rezervacia_stol_datum
    ON Rezervacia (cislo_stola, datum, stav);

ANALYZE Rezervacia;

EXPLAIN (ANALYZE, BUFFERS)
```

```sql
SELECT r.id, r.id_zakaznika, r.cislo_stola, r.datum,
       r.cas_zaciatku, r.cas_konca, r.stav
FROM Rezervacia r
WHERE r.cislo_stola = 8
  AND r.id_zakaznika = 320
  AND r.datum = DATE '2026-05-03'
  AND TIME '13:15' BETWEEN r.cas_zaciatku AND r.cas_konca
  AND r.stav = 'potvrdena'
LIMIT 1;
```

Výrez plánu s indexom:

```
Limit (actual time=0.015..0.015 rows=1 loops=1)
  Buffers: shared hit=3
  -> Index Scan using idx_rezervacia_stol_datum on rezervacia r
        Index Cond: ((cislo_stola = 8)
                    AND (datum = '2026-05-03'::date)
                    AND (stav = 'potvrdena'::stav_rezervacie))
        Filter: (id_zakaznika = 320
                AND '13:15'::time BETWEEN cas_zaciatku AND cas_konca)
Planning Time: 0.567 ms
Execution Time: 0.032 ms
```

Po vytvorení indexu sa plán zmenil zo sekvenčného skenu na `Index Scan`. Databáza už nehľadala rezerváciu v celej tabuľke, ale použila indexovú podmienku nad stĺpcami `cislo_stola`, `datum` a `stav`. Aj na relatívne malom seede klesol čas vykonania z približne `0.254 ms` na `0.032 ms`. Zvyšné podmienky na zákazníka a časový interval sa aplikujú už iba na kandidátov nájdených cez index.

8 Zhodnotenie

Najnáročnejšou časťou bolo zosúladiť fyzický model s procesmi tak, aby procesy neboli iba jednoduché SELECT dotazy, ale pokrývali reálnu prevádzkovú logiku. Pri procese 1 bolo dôležité vytvoriť objednávku čisto v SQL bez PL/pgSQL, ale zároveň zachovať validačné kroky a atomickosť zápisu.

Pri implementácii sa ukázalo, že niektoré biznis pravidlá nie je vhodné riešiť obyčajným `CHECK` obmedzením. Typickým príkladom je prekrývanie rezervácií, ktoré vyžaduje kontrolu voči iným riadkom. Toto pravidlo preto riešime validačným dotazom v procese.

Indexy sme nenavrhovali plošne na všetky stĺpce. Zamerali sme sa na časti dopytov, ktoré sa opakujú v procesoch: hľadanie rezervácie podľa stola a dátumu, načítanie položiek objednávky, filter zaplatených faktúr a rozsahový filter objednávok podľa dátumu.

AI nástroje sme použili ako pomoc pri kontrole konzistentnosti textu, formulácii reportu a prepisovaní SQL procesu do čistej deklaratívnej podoby. Výsledné SQL a model sme skontrolovali voči zadaniu a upravili podľa reálnych tabuliek v projekte.